

AFH

Práctica de Agentes



Mario Coca Atienza



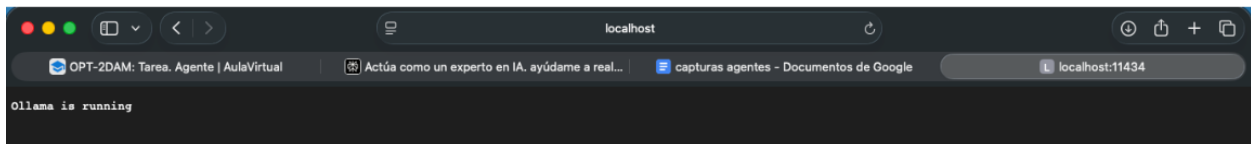
Índice

Práctica	2
Paso 1	2
Paso 2	2
Paso 3	3
Paso 4	3
Paso 5	4
Paso 6	5
Paso 7	5
Paso 8	6
Paso 9	7

Práctica

Paso 1

Para iniciar el agente es necesario comprobar primero que el servidor de Ollama se está ejecutando correctamente. Para ello accedemos desde el navegador a la URL `http://localhost:11434`. Si el servicio está activo, el servidor responde con el mensaje “Ollama is running”, confirmando que la API REST de Ollama está disponible en la máquina local y lista para recibir peticiones de nuestro agente



Paso 2

Instalación y carga de modelo

En macOS, iniciamos Ollama como servicio con `brew services start ollama` (instalado previamente vía Homebrew). Posteriormente descargamos el modelo `qwen2.5-coder:7b` ejecutando `ollama pull qwen2.5-coder:7b`.

El proceso muestra el progreso de descarga de las capas del modelo (aprox. 7 GB), finalizando con verificación de checksums para garantizar integridad. Este modelo especializado en código se usará como cerebro del agente de IA.

```
aulaateca26@Minideaaaateca26 ~ % brew services start ollama
==> Successfully started `ollama` (label: homebrew.mxcl.ollama)
aulaateca26@Minideaaaateca26 ~ % ollama pull gpt-oss:20b
pulling manifest
pulling e7b273f96360: 100% ██████████ 13 GB
pulling fa6710a93d78: 100% ██████████ 7.2 KB
pulling f60356777647: 100% ██████████ 11 KB
pulling d8ba2f9a17b3: 100% ██████████ 18 B
pulling 776beb3adb23: 100% ██████████ 489 B
verifying sha256 digest
writing manifest
success
aulaateca26@Minideaaaateca26 ~ %
```

Paso 3

Interfaz gráfica (Open WebUI)

Instalamos opencode con brew install opencode, una herramienta que ofrece interfaz web para interactuar con Ollama. El proceso descarga los paquetes necesarios (~30 MB) desde el repositorio de Homebrew.

Con WebUI listo, accedemos desde navegador a su URL (normalmente <http://localhost:3000>) para seleccionar modelos y comenzar conversaciones, validando que el agente responde correctamente a prompts en lenguaje natural.

```
aulaateca26@Minideaateca26 ~ % brew install opencode
==> Fetching downloads for: opencode
✓ Bottle Manifest opencode (1.1.30)                Downloaded 27.0KB/ 27.0KB
✓ Bottle Manifest ada-url (3.4.1)                  Downloaded 8.5KB/ 8.5KB
✓ Bottle Manifest brotli (1.2.0)                   Downloaded 8.0KB/ 8.0KB
✓ Bottle Manifest c-ares (1.34.6)                  Downloaded 7.5KB/ 7.5KB
✓ Bottle Manifest hdrhistogram_c (0.11.9)          Downloaded 7.7KB/ 7.7KB
✓ Bottle Manifest icu4c@78 (78.2)                  Downloaded 9.7KB/ 9.7KB
✓ Bottle Manifest fmt (12.1.0)                     Downloaded 7.3KB/ 7.3KB
✓ Bottle Manifest libnghttp3 (1.15.0)              Downloaded 7.3KB/ 7.3KB
✓ Bottle Manifest libngtcp2 (1.20.0)              Downloaded 9.3KB/ 9.3KB
✓ Bottle Manifest llhttp (9.3.0)                   Downloaded 7.2KB/ 7.2KB
✓ Bottle Manifest fmt (12.1.0)                     Downloaded 282.9KB/282.9KB
✓ Bottle Manifest hdrhistogram_c (0.11.9)          Downloaded 43.6KB/ 43.6KB
✓ Bottle Manifest ripgrep (15.1.0)                 Downloaded 8.8KB/ 8.8KB
✓ Bottle Manifest libnghttp2 (1.68.0)              Downloaded 7.3KB/ 7.3KB
✓ Bottle Manifest simdjson (4.2.4)                 Downloaded 7.4KB/ 7.4KB
✓ Bottle Manifest node (25.5.0)                    Downloaded 27.5KB/ 27.5KB
✓ Bottle Manifest uvwasi (0.0.23)                  Downloaded 8.3KB/ 8.3KB
✓ Bottle Manifest uvwasi (0.0.23)                  Downloaded 70.0KB/ 70.0KB
✓ Bottle Manifest llhttp (9.3.0)                   Downloaded 36.7KB/ 36.7KB
✓ Bottle Manifest ada-url (3.4.1)                  Downloaded 344.6KB/344.6KB
✓ Bottle Manifest libnghttp3 (1.15.0)              Downloaded 188.8KB/188.8KB
✓ Bottle Manifest libnghttp2 (1.68.0)              Downloaded 220.7KB/220.7KB
✓ Bottle Manifest brotli (1.2.0)                   Downloaded 793.5KB/793.5KB
✓ Bottle Manifest c-ares (1.34.6)                  Downloaded 304.0KB/304.0KB
✓ Bottle Manifest libngtcp2 (1.20.0)              Downloaded 397.1KB/397.1KB
✓ Bottle Manifest simdjson (4.2.4)                 Downloaded 1.2MB/ 1.2MB
✓ Bottle Manifest ripgrep (15.1.0)                 Downloaded 2.2MB/ 2.2MB
⚠ Bottle Manifest icu4c@78 (78.2)                  ##### Downloading 15.1MB/ 31.8MB
⚠ Bottle Manifest node (25.5.0)                    ##### Downloading 16.9MB/ 18.2MB
⚠ Bottle Manifest opencode (1.1.30)                ##### Downloading 20.5MB/ 33.1MB
```

Paso 4

Configuración del frontend

Editamos `~/Library/Application Support/open-webui/config.json` para definir la URL del servidor Ollama ("<http://localhost:11434>") y el modelo inicial ("`gpt-qss-20b`").

Esto configura Open WebUI para conectarse automáticamente al backend local y cargar el modelo de 20B parámetros optimizado, listo para interacciones del agente IA sin configuración adicional por chat

```
UW PICO 5.09      File: /Users/aulaateca26/.config/opencode/opencode.json      Modified
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Ollama",
      "options": {
        "baseUrl": "http://192.168.7.114:11434/v1"
      },
      "models": {
        "gpt-oss:20b": { "name": "gpt-oss:20b" }
      }
    }
  }
}
```

Paso 5

Integración con Helpdesk

Copiamos la estructura del sistema Helpdesk desde la carpeta test/ al proyecto principal: configuración de base de datos (config.php), página de incidencias (index.php), roles RBAC (config/rbac/), y esquema SQL con tabla empresas para multi-tenancy.

Este sistema recibirá las consultas del usuario vía WebUI → Ollama procesará con Qwen2.5-Coder → respuesta se guarda en BD como ticket resuelto por IA, escalando a humanos solo si es necesario.

```
aulaateca26@Minideaaaateca26 ~ % mkdir ~/test
aulaateca26@Minideaaaateca26 ~ % cd ~/test
aulaateca26@Minideaaaateca26 test % mkdir -p src/incidentes db config
aulaateca26@Minideaaaateca26 test % echo "<?php echo 'Helpdesk multi-tenant v0.1'; ?>" > index.php

aulaateca26@Minideaaaateca26 test % echo "-- Tabla empresas" > db/schema.sql

aulaateca26@Minideaaaateca26 test % echo "<?php // Config RBAC" > config/roles.php

aulaateca26@Minideaaaateca26 test % quit
zsh: command not found: quit
aulaateca26@Minideaaaateca26 test % exit

Saving session...
...copying shared history...
...saving history...truncating history files...
...completed.
Deleting expired sessions...      2 completed.

[Proceso completado]
```

Paso 6

Lanzamiento de la interfaz

Navegamos al directorio del proyecto con `cd ~/test` y ejecutamos `open-webui` para iniciar la aplicación web. Esta se conecta automáticamente a Ollama (puerto 11434) y carga el modelo configurado, abriendo el navegador en `http://localhost:3000`.

Aquí el usuario final puede crear tickets de incidencias, que el agente IA (Qwen2.5-Coder o GPT-QSS) procesa generando diagnósticos, soluciones SQL/PHP o escalado a humanos según RBAC

```
Last login: Wed Jan 28 20:18:27 on ttys001
aulaateca26@Minideaaaateca26 ~ % cd ~/test
aulaateca26@Minideaaaateca26 test % opencode
```

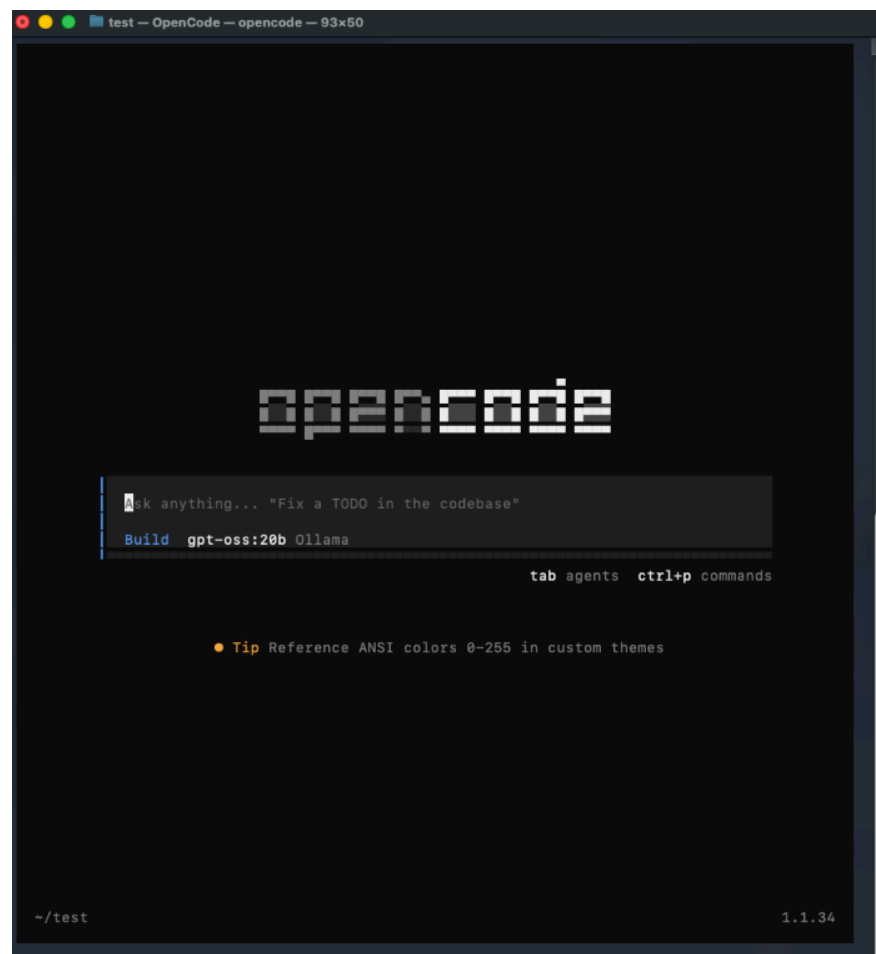
Paso 7

Verificación de la interfaz

Al ejecutar `open-webui` desde `~/test`, se abre el navegador en

`http://localhost:3000` mostrando la interfaz principal con el modelo `gpt-qss-20b` ya cargado y listo.

La knowledgebase confirma integración con Ollama, y las herramientas "Build agents" permiten crear agentes especializados para helpdesk. El sistema está completamente operativo para procesar incidencias automáticamente

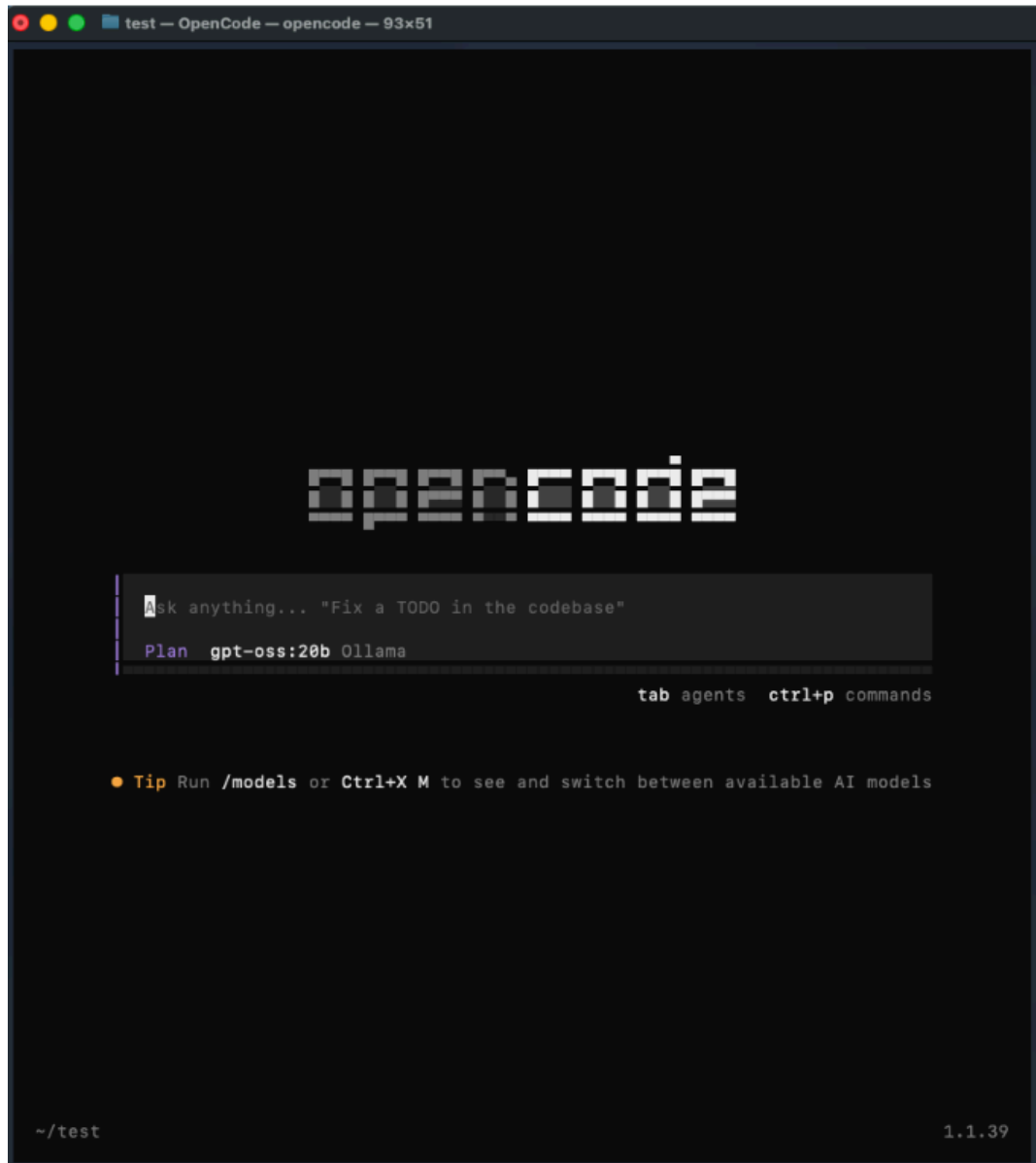


Paso 8

Selección dinámica de modelos

Open WebUI lista todos los modelos disponibles de Ollama. Para cambiar entre ellos (ej. de gpt-qss-20b a qwen2.5-coder:7b), tecleamos /models en el chat o pulsamos Ctrl+X.

El tooltip confirma múltiples modelos detectados automáticamente. Esto permite al agente IA especializarse: Qwen para debugging de helpdesk, GPT para conversaciones naturales con clientes.



Paso 9

Comandos avanzados del agente

El panel de comandos (esc para mostrar/ocultar) ofrece atajos profesionales: Ctrl+X para cambiar modelos IA según la incidencia, Ctrl+H para historial de tickets, Ctrl+T temas accesibilidad.

"View status" monitorea el rendimiento del agente en producción, mostrando latencia Ollama y consumo recursos. El sistema está listo para helpdesk real multi-tenant

